

Making Your Data Ready for Agentic AI

For thirty years we built data systems for human analysts, who supply the context, judgment, and skepticism to work around data that's incomplete or wrong. Autonomous agents supply none of that. They act on whatever they're handed, confidently. For data to be AI-ready we need to build a series of layers: a data foundation that makes data trusted, a context layer to apply proper meaning, and an access layer that supports and controls how agents operate on that data. While doing this we need continuous attention to observability that ensures the data is properly governed and we have an auditable trace of its use in decision-making.

27 August 2026



Pramod Sadalage

Pramod is a Distinguished Engineer at Thoughtworks leading the Data Engineering and Architecture service for North America, where he helps clients with particularly challenging data needs, which require new technologies and techniques. In the early 00's he developed techniques to allow relational databases to be designed in an evolutionary manner based on version-controlled schema migrations. He has been an author on five books on software development, including "Software Architecture: The Hard Parts" and "Refactoring Databases"

CONTENTS

- [The consumers of your data are changing](#)
- [What "AI-ready" has to mean now](#)
- [Data Contracts and Quality: Agents Can't Smell Bad Data](#)
 - [Agents treat every value as truth](#)
 - [Schema is law: data contracts as code](#)
 - [The quarantine pattern](#)
 - [Medallion architecture for agents](#)
 - [The same rules for unstructured data](#)
 - [Confidence-threshold routing](#)
 - [Where to start](#)
- [Traceability and Governance: Auditing Autonomous Agents](#)
 - [The audit gap](#)
 - [Agentic lineage](#)
 - [The regulatory teeth are real](#)
 - [Staged autonomy](#)
 - [Delegated access and just-in-time credentials](#)



Prem Chandrasekaran

Premanand (Prem) Chandrasekaran is a Market Tech Director at Thoughtworks, where he leads technology strategy and delivery across a portfolio of clients and guides teams through complex, high-stakes engineering decisions. He stays deliberately hands-on, designing and building production-grade systems alongside the teams he leads. His current focus is on how AI can augment the software development lifecycle, from architecture and design to coding, testing, and operations, so that developers can move faster without sacrificing clarity, safety, or maintainability.

◀ ENTERPRISE ARCHITECTURE

◀ DATA ANALYTICS

◀ GENERATIVE AI

Where to start

The Context Layer: Teaching Agents What Your Data Means

Your agent doesn't know what "revenue" means

What the context layer is

Metrics as code

Same question, very different SQL

How agents use it

Where to start

Traversing the domain model: knowledge graphs

From Searchable to Actionable: Agent-Ready Data Access

Your agent can read, but it can't act

The data access spectrum

Three primitives, one protocol

Antipattern: naive API-to-MCP conversion

What a capability declares

Retrieved text informs, it never gates

End to end: the PO payment scenario

Where to start

The AI-ready data stack

Who owns all this?

Where do you stand?

Four things to start on

SIDEBARS

Explainer: the words people use for semantic layers

There's a lot of excitement right now about agent frameworks, orchestration patterns, and protocols. All of it matters, but almost none of it delivers value if you skip the data layer. Before any agent framework can produce useful outcomes, your data has to be in a shape that a machine can consume, trust, and act on. In this article, we discuss what your data needs to look like for agentic AI to derive value from it.

We've spent quite a bit of time building data architectures for the human consumer. We're about to hand those architectures to a very different kind of consumer, and most of them aren't ready for it.

The consumers of your data are changing

For over three decades, we've been building data systems for humans. Dashboards, reports, analyst queries, all of it designed around a person sitting in front of a screen. And it worked, because humans bring an enormous amount of implicit context, along with the curiosity to track down whatever they're missing from the people around them.

A human analyst knows what “revenue” means in your particular organization. They know which tables to query and which ones to avoid. They notice when a number looks off, when a total is suspiciously round, when a date falls on a public holiday, or when a price seems too low. That instinct is doing a large amount of invisible context and knowledge work.

A human hesitates at data that looks wrong; an agent acts on it anyway

Agents have none of it. They can't lean on the tribal knowledge and pattern recognition people accumulate over years, so they need context made explicit, access in real time, and quality they can rely on. And the difference that matters most is this: when the data feels wrong, a human double-checks; an agent confidently acts on it. That behavioral gap is what the rest of this discussion is built around.

What “AI-ready” has to mean now

For a human consumer, the data only had to be *good enough*; the analyst did the rest. The meaning, the sanity check, and the judgment about whether a number could be trusted all lived in a person's head. When the same data is handed to an agent, every bit of that implicit labor has to move into the data itself. That shows up as five attributes, each the flip side of something a human used to do for free.

- **Trusted:** a person pauses at a number that feels wrong; an agent acts on it. The confidence a human used to supply has to be built in, so the data must be accurate, fresh, and validated before the agent ever sees it.
- **Contextual:** a person knows your “revenue” figure already has returns taken out, and that your fiscal year starts in February; an agent has to be told both. Meaning that used to live in someone's head has to be made explicit in the data.
- **Traceable:** when a person decides, they can explain why afterward; when an agent decides in 30 seconds, that reasoning is gone unless you capture it as it happens. You have to be able to reconstruct what the agent did and why.
- **Governed:** a person's access is bounded by their role and their judgment; an agent's has to be bounded by design. Access must be scoped, controlled, and auditable.
- **Operational:** a person reads a dashboard and then goes and does something; an agent has to be *able* to do the something. The data can't just be readable, it has to be actionable.

All five come down to the same idea. Each is a job humans used to do without thinking, now pushed into the data itself. Miss one, and the agent won't degrade gracefully the way a person would. It fails confidently.

None of these attributes builds itself. The rest of the article works through four topics that do, roughly in the order you should tackle them.

1. Data Contracts and Quality makes data *Trusted*. We start here, because a single wrong fact poisons every layer built on top of it.
2. Traceability and Governance records why an agent acted and bounds what it can reach, making data *Traceable* and *Governed*.
3. The context layer encodes what your metrics and entities mean, making data *Contextual*.
4. From Searchable to Actionable lets agents query live systems and write back, making data *Operational*.

We'll take them one topic at a time, and show what it takes to build each attribute in. Work through all four, and the five attributes stop being abstract goals. They become something you can engineer, turning ordinary data into AI-ready data.

Data Contracts and Quality: Agents Can't Smell Bad Data

Humans have a smell test for bad data. They notice when a number looks off, when a date makes no sense, or when a price seems wrong. Agents have no such instinct. As Simon Willison puts it, language models are gullible, they believe whatever they are handed and act on it. Feed an AI agent a wrong value, and it won't pause to wonder, it will use the number and produce a confident, wrong answer. Without trusted data, nothing else in agentic AI works, so this is where we begin.

Agents treat every value as truth

Consider a concrete scenario. A pricing agent is asked for the current price of Product X. Yesterday, the price was updated from \$49.99 to \$59.99. But the agent's data source hasn't refreshed, it still shows the old number.

The agent doesn't hesitate, it retrieves \$49.99, quotes the customer, the customer buys, and the company loses \$10 on every unit sold. Every step the agent took was technically correct. It followed its workflow perfectly. The *data* it accessed was the problem.

The leaders most confident their data is AI-ready also name data readiness their biggest barrier

A human sales rep would have paused: “Wait, didn't we update this last week?” They'd double-check. They have institutional memory and a feel for when something's off. The agent has neither. Errors don't trigger warnings; they cascade silently through the workflow. And this isn't a rare edge case. In the 2026 State of Data Integrity and AI Readiness report, Precisely and Drexel University's LeBow College of Business surveyed 505 data and analytics leaders, of whom 87% believed their data was ready for AI, yet 43% named data readiness as the single biggest barrier to getting value from it. That gap between confidence and readiness is the organization-level version of the pricing agent, sure of itself and wrong. A separate KPMG Global AI Pulse survey of 2,145 leaders

points the same way, with nearly half of executives now seeing AI's costs exceed its benefits. Most enterprises are one stale field away from the scenario above.

Schema is law: data contracts as code

So how do you prevent AI agents from accessing bad or stale data? The answer is *data contracts*, treating schema as law, not a polite suggestion.

This reverses a decade of “schemaless is flexible” thinking, for human consumers, loose schemas are merely inconvenient, while for AI agents, they're dangerous. A data contract, written in the Open Data Contract Standard, the format the Data Contract CLI uses (and recommended in Thoughtworks tech radar 33), defines the rules explicitly. A `product_pricing` contract might specify:

- Properties with strict logical types.
- A quality rule that price must be greater than zero.
- A quality check on currency that rejects anything outside USD, EUR, or GBP.
- Critically, a *freshness SLA*, pricing data must have been refreshed within the last 24 hours.

In the Open Data Contract Standard, that contract is shown below.

```

apiVersion: v3.1.0
kind: DataContract
id: product-pricing
name: Product Pricing
version: 1.0.0
status: active
schema:
  - name: product_pricing
    physicalType: table
    properties:
      - name: product_id
        logicalType: string
        physicalType: varchar(64)
        required: true
        unique: true
        primaryKey: true
        primaryKeyPosition: 1
      - name: price
        logicalType: number
        physicalType: decimal
        required: true
        quality:
          - type: sql
            description: Every price must be greater than zero
            query: SELECT min({property}) FROM {object}
            mustBeGreaterThan: 0
      - name: currency
        logicalType: string
        physicalType: varchar(3)
        required: true
        quality:
          - type: sql
            description: Currency must be a supported ISO code
            query: SELECT count(*) FROM {object} WHERE {property} NOT IN ('USD', 'EUR',
'GBP')
            mustBe: 0
      - name: ingested_at
        logicalType: timestamp
        physicalType: timestamp
        required: true
slaProperties:
  # the rule that would have caught the stale-price scenario
  - property: latency
    value: 24
    unit: h
    element: product_pricing.ingested_at

```

Enforcement happens along three dimensions.

- Schema enforcement ensures types and constraints are respected and made explicit by the contract.
- Freshness SLAs define the maximum acceptable staleness per dataset, nightly batch updates aren't enough when an agent answers in real time. Key the SLA to when the data was last successfully loaded, not when a value last changed, so that steady data isn't flagged as stale and a stalled pipeline can't masquerade as fresh.
- Quality gates validate contracts in CI/CD, blocking deployments when they fail.

Notice how this changes the earlier pricing scenario, it prevents it by design. If the pricing data hasn't been refreshed in 24 hours, the contract is violated *before the agent ever sees the data*.

The quarantine pattern

Defining a contract is one thing. What happens when data violates it? You need a circuit breaker and that's the quarantine pattern.

The flow works like this. Raw data arrives from source systems, APIs, databases, streams. Before it enters the agent accessible data store, it passes through a contract validation gate that checks three things, does it match the schema, is it within the freshness SLA, and does it pass the quality rules?

If it passes all three, it flows into the certified, agent ready tier. If it fails any one of them, it's quarantined, routed to a dead letter queue for human review, with alerts fired.

*Bad data lands in a dead-letter queue,
never in front of the agent*

The point is that the agent never sees the bad data. It doesn't get poisoned by stale prices or corrupted embeddings. In the pricing scenario, if the `ingested_at` timestamp is older than 24 hours the contract is violated and the record is quarantined, so when asked about the price the agent says, "I don't have current pricing data" rather than confidently quoting the wrong number. That is a far better failure mode. And it's a job for the data architecture, not the model. A better model won't rescue you from bad data.

Medallion architecture for agents

A medallion architecture is an analytical data design pattern for organizing data in a lakehouse, popularized by Databricks.

Bad data gets quarantined, but where does the *good* data go? That's what the medallion architecture organizes, and its first three tiers are well established:

- **Bronze:** raw, immutable ingestion. You keep everything for audit trail and lineage.
- **Silver:** validated and deduplicated. Schema is applied, data contracts are enforced, and this is where the quarantine pattern lives.
- **Gold:** certified. This is what the semantic model compiles against, access is governed, and metrics are trusted.

For agentic architectures, a fourth tier is worth adding. In **Adaptive Gold**, shown below, agents curate data rather than only consume it. They watch their own query patterns and materialize the combinations they keep reaching for, building warehouse views from real usage. This is not speculative. At DataHub's CONTEXT 2025 summit, Apple described agents acting as “digital stewards” of its data catalog, continuously scanning metadata, flagging gaps, and proposing updates. Apple's agents curate the *catalog*; Adaptive Gold points the same pattern at the *datasets*. That's the next step. Early days, but worth trying once your agents' query patterns have settled.

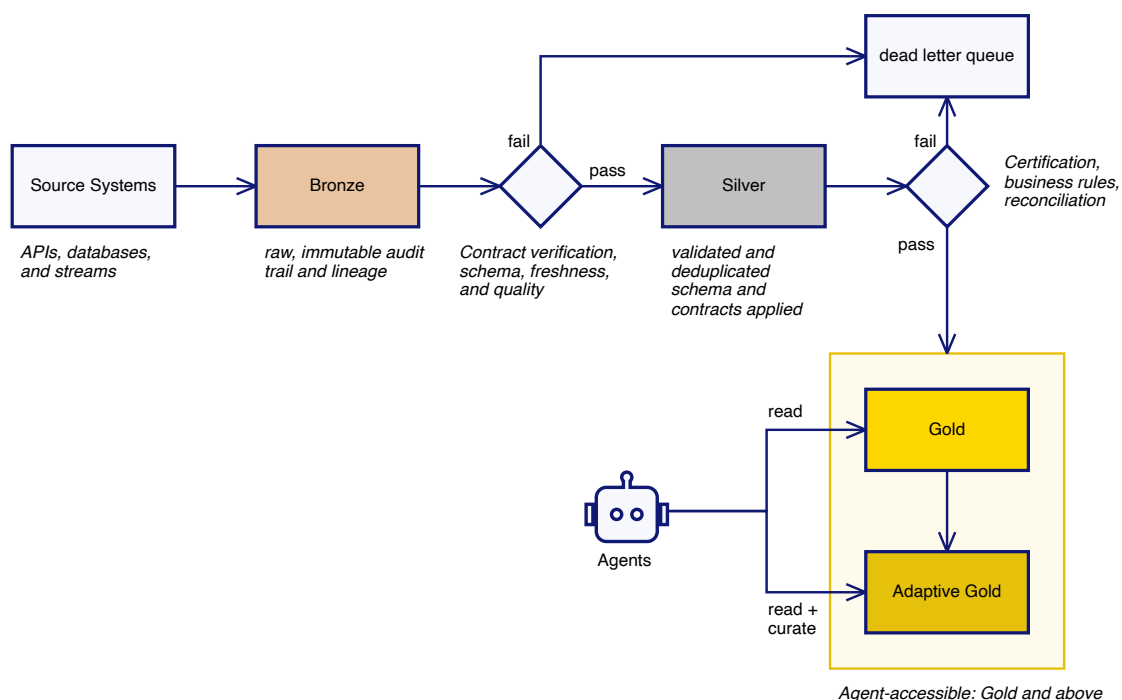


Figure 1: Medallion tiers for agents: data flows from raw Bronze through validated Silver to certified Gold and agent curated Adaptive Gold, while agents are restricted to Gold and above.

*Bronze and Silver are for humans;
agents see only Gold and above*

The key architectural principle is that agents should only access Gold tier or above. Bronze and Silver exist for lineage, debugging, and human investigation. Exposing raw or partially validated data to agents invites the pricing problem back in.

The same rules for unstructured data

Everything so far has looked like a table, prices, currencies, timestamps, but most of what agents consume isn't tabular. It's documents, wikis, PDFs, and support tickets, chunked and embedded into a vector store for retrieval. If your agents do RAG, this is the data they run on, and it needs the same trust guarantees, even though you can't write $\text{price} > 0$ on a paragraph. The patterns carry over, only the quality dimensions change.

The stale-price scenario has a twin here. A policy document gets updated, but the vector index isn't re-embedded, so the agent retrieves the old version and answers confidently from it, the same failure as the stale price, only now it's an embedding rather than a row. The *freshness* SLA carries over, but be precise about what the clock measures, the point isn't when the content last changed, it's when the index was last successfully rebuilt against its sources. A 24-hour SLA means the re-indexing job must have completed within the last 24 hours, if it hasn't, the index is stale and quarantined even when nothing appears to have changed, because a silently failed indexer is exactly when you can't tell whether something did. That one heartbeat catches both the *updated but unindexed* document and the pipeline that quietly stopped.

Contracts move from the content to the surrounding metadata. You can't constrain the prose, but you can require that every chunk carry a source, a version, a timestamp, and an access scope, and reject anything that doesn't. That metadata is also what makes retrieval traceable and governable later.

Quality gates get checks suited to text, reject empty or truncated chunks, catch near-duplicate documents that skew retrieval, flag failed extractions and OCR garbage, and watch for embedding drift. A malformed or empty embedding warps similarity search,

so it never reaches the store, for the same reason a bad price never reaches the agent, a warped index makes the agent retrieve confidently wrong content.

Whether the data is a priced row or an embedded paragraph, the job is identical. The architecture has to smell what's bad before the agent does.

Confidence-threshold routing

Contracts, quarantine, and the medallion architecture handle the clear cases. But there's a gray area, data that isn't clearly bad, but isn't fully trustworthy either. That's where confidence-threshold routing comes in, bridging full autonomy and full human control.

The agent processes a request and assesses data quality signals, and checks not just *model* confidence, but *data-level* signals like freshness, completeness, and consistency. If confidence is at or above the threshold (say 85%), the agent proceeds autonomously. Below it, the agent defers to a human. The threshold is configurable per use case, for example, pricing might demand 90%, while an internal FAQ is fine at 70%.

Let's return to the pricing scenario one last time. The price data is three days stale; the freshness SLA says 24 hours. The SLA violation automatically drives the confidence score below the threshold, regardless of how confident the model itself feels about its answer. The agent should respond by pulling a human in:

"I'm not confident this price is current. Routing to a human for verification."

Data quality signals should drive the threshold, not just the model's own confidence

In other words, data quality signals should drive the threshold, not just the model's own confidence. A model can be sure of a stale answer, and the freshness SLA overrides that misplaced certainty.

The hard part is turning those quality signals into a single score and weighing it against the model's own confidence. That's an open design problem, not a solved one. Start with a hard gate rather than a smooth composite. Any contract or SLA breach forces a

human, regardless of how the other signals look. Add weighted scoring later, and only once you can show it beats that simple rule.

Where to start

You don't have to build all of this at once, and most teams can't. Contracts, quarantine gates, a medallion architecture, and confidence-threshold routing are a lot to stand up in one go. The good news is that they're additive, each one lowers risk on its own, and you can layer in the rest over time. Begin with the highest leverage moves and expand from there.

1. Define freshness SLAs for every dataset agents touch. The same dataset can have different freshness requirements per consumer, such as a pricing table that's fine on nightly batches for a dashboard may need near real time updates when a quoting agent depends on it.
2. Implement quarantine gates. Validate against contracts before data enters agent accessible storage. Start with your highest risk datasets such as pricing, inventory, customer records.
3. Start with the Data Contract CLI. Bring contract governance into CI/CD, define contracts as YAML, validate automatically, block deployments on failure. Treat data contracts with the same rigor you'd give an API contract.
4. Add confidence threshold routing. When quality signals drop below a threshold, defer to a human. Start high (around 90%) and adjust downward as you build trust and track accuracy.

We've made data trustworthy. But when agents act autonomously on that data, who's watching?



Traceability and Governance: Auditing Autonomous Agents

Even with perfect data, autonomous action raises a harder question, when a regulator asks why the agent did what it did, can you answer? Traditional systems record what happened. Agentic ones have to explain why. That shift, from *what* to *why*, is where governance gets hard.

The audit gap

Picture a bank running agentic AI for trade finance, where the governance architecture is the real innovation.

An agent processes a letter of credit. It checks KYC data, verifies the customer isn't on a sanctions list, evaluates the credit terms, and approves a \$2.4 million transaction, all in about 30 seconds. Six months later, a regulator asks a simple question, “Why was this approved?”

*Traditional audit logs can tell you **what** happened, but they can't tell you **why**.*

Traditional audit logs can tell you *what* happened, which tables were queried, at what time, by which service account. What they can't tell you is *why*. Why did the agent check the sanctions list before the credit terms? Why did it approve despite a minor documentation discrepancy? What alternatives did it consider and reject? The gap between “what” and “why” is where regulatory risk arises, and the EU AI Act's Article 12 requires high-risk systems to keep automatic logs for exactly this reason, so the “why” can be reconstructed after the fact. Closing that gap is what agentic lineage is for.

Agentic lineage

The way to close this audit gap is *agentic lineage*, an extension of traditional data lineage. Where traditional lineage tracks which sources were accessed, agentic lineage tracks why the agent decided to access X, because it found Y in source Z.

Concretely, for the trade finance case, a single trace represents the end-to-end workflow of processing letter of credit LC-4892. Within that trace, each span is an individual step:

- Span 1: retrieved customer KYC data from the compliance database, result: verified.

- Span 2: checked the sanctions list via the OFAC API, result: clear.
- Span 3: evaluated credit terms against the policy engine, result: within limits.
- Final span: the decision, APPROVE, with a 94% confidence score and the full reasoning chain attached.

This is exactly what a regulator needs. Not “the agent accessed the compliance database at 14:32:07 UTC” but “the agent checked KYC first, then sanctions, then credit terms, and approved because all three passed.” The *traces and spans model* is borrowed directly from distributed systems observability, so engineers already understand the mental model from tools like [Jaeger](#) and [Zipkin](#). For the agentic equivalent, [Langfuse](#), [Arize Phoenix](#), and [OpenTelemetry](#) for AI are the emerging choices. All three feature on the Thoughtworks Technology Radar, OpenTelemetry at Adopt, Langfuse at Trial, and Arize Phoenix at Assess.

The regulatory teeth are real

This isn't a theoretical exercise. The EU AI Act is the most specific regulation on the books. [Article 12](#) requires high-risk AI systems to automatically log events over their lifetime so their operation can be traced, and [Article 19](#) requires providers to keep those logs for at least six months. Breaching these record-keeping obligations falls in the Act's middle penalty tier, up to €15 million or 3% of global annual turnover, whichever is higher. For a large company, even 3% of global turnover runs into the hundreds of millions.

Together, Articles 12 and 19 translate into three obligations for your architecture:

- Automatically log events across the system's lifetime, enough to trace how it operated, not just isolated timestamps.
- Retain those logs for at least six months, which means your observability infrastructure has to handle long-term storage.
- Be able to reconstruct the “why” after the fact. The law mandates the logs; making them answer a regulator's question is on you. That means capturing the full reasoning chain, which sources were consulted, what logic was applied, and which alternatives the agent weighed and rejected.

The EU is furthest ahead, and for now no other jurisdiction has a law quite like it. But you don't have to bet on where regulation lands to see the point. Sooner or later something will force the question of why an agent did what it did, whether that's a regulator, an auditor, a customer disputing a decision, or just your own team trying to

debug one. The safe assumption isn't that a particular law is coming, it's that you'll want to answer that question regardless. A system you can't explain is one you can't fully trust, defend, or fix.

Staged autonomy

Knowing you need audit trails is one thing; rolling this out safely is another. You don't deploy an agent with full autonomy on day one, any more than you'd hand a brand new employee unrestricted access. Autonomy is earned in stages:

Stage	Agent	Human	Monitoring
Shadow Mode	Recommends actions	Reviews recommendation and executes if appropriate	All recommendations are logged to track accuracy over time
Supervised	Prepares action and waits for approval	Reviews action and approves or denies	All proposed actions and human decisions are logged
Autonomous with guardrails	Agent acts within defined boundaries (best drawn by reversibility, not transaction size)	Defines guardrails	All actions logged, alerts fired on exceptions
Full autonomy	Agent carries out all actions	Spot checks	Continuous, by other agents and humans

You wouldn't give a new hire the corporate credit card on day one. They start with purchase requests, graduate to supervised spending, and eventually earn a card with limits. Agents should earn trust the same way.

Promotion up this ladder should turn on evidence, not a hunch. That means testing an agent before each step, not only watching it in production. Agents are hard to test. They're nondeterministic, costly to call, and act through tools with real side effects. So teams mock or replay the tool and model interactions so tests run deterministically in CI. They score the agent's decisions with evals rather than calling live services on every run. Building that harness is a discipline of its own, and beyond the scope of this article.

Delegated access and just-in-time credentials

As agents earn autonomy, the question becomes, what permissions should they hold? Three security patterns matter most here.

- **Delegated Access:** When Alice asks the agent to check her account, the agent should act *with Alice's permissions*, not through a broad service account that can see every customer's data. Shared service accounts destroy attribution. When a regulator asks “who accessed this customer's data?”, “the service account” tells you almost nothing. With delegated access, the answer is “Alice's agent, acting on Alice's behalf, with Alice's permissions.”
- **Just-in-time Credentials:** Instead of a persistent API key that never expires, issue a short-lived token for each specific task. The agent needs to check the sanctions list? Issue a token scoped to OFAC API read access for that specific customer, valid for five minutes. When the task completes, the token expires. No standing credentials sitting around waiting to be compromised.
- **Least Privilege:** The agent gets the minimum access the task requires. Processing a letter of credit doesn't need reach into HR systems or marketing data.

Together, these three patterns address the attribution and scope challenges that undermine many current agentic deployments.

They also defend against the sharpest security risk in agentic systems. Simon Willison calls it the lethal trifecta, an agent turns dangerous the moment it holds all three of access to private data, exposure to untrusted content, and a way to communicate externally. Put those together and a single poisoned document or web page can hijack the agent through prompt injection and quietly exfiltrate whatever it can reach. Delegated access, *just-in-time* credentials, and least privilege shrink how much a hijacked agent can reach, breaking the trifecta. Later we add a second cut at the same problem, keeping retrieved text out of the authorisation path entirely, so that a poisoned document cannot grant a permission in the first place.

Where to start

Of the four topics, this is the one where going slowly is the right instinct. But separate two things that are easy to conflate. Autonomy is earned in stages, so nobody expects you to grant it all at once. Observability is not staged at all. It goes in from day one, at full strength, whatever the autonomy level, because retrofitting it onto a running

system is painful. What you build on top can stay deliberately conservative; the instrumentation underneath cannot.

- Instrument from day one. Of everything here, this is the one to do first, adding observability after deployment is far harder. Every agent workflow should emit traces with spans for each step, including reasoning and sources consulted. The tracing pattern here is well established, so lean on a proven tool (like OpenTelemetry) rather than building your own.
- Start in shadow mode. Lowest risk, highest learning. Agents recommend, humans decide. You build the audit trail before you need it for compliance and measure accuracy before granting autonomy.
- Implement delegated access. Agents inherit the invoking user's permissions and use *just-in-time* credentials with short expiry windows. No persistent tokens.
- Build to be explainable. Whether or not a regulator ever asks, an audit trail that answers “why” is what lets you debug a bad decision, defend a good one, and trust the system enough to widen its autonomy. Wire it in now, it's far harder to add later.

Semantic layers bridge the institutional knowledge gap between agents and human analysts, building on trusted data and auditable actions provided by the earlier topics.

The Context Layer: Teaching Agents What Your Data Means

Semantic layers provide the explicit context AI agents need when they become the primary consumers of data, context that human analysts carry implicitly, based on years of experience.

Your agent doesn't know what “revenue” means

Ask an agent, “What was Q3 revenue for Product X?” A human analyst knows precisely what to do, which table to query, whether revenue means gross or net, what Q3 maps

to in your fiscal calendar. They absorbed all of it over years of institutional knowledge, tribal docs, and Slack threads.

The agent has none of it. It doesn't know which joins connect products to orders to revenue, or that your fiscal calendar starts in February. With that context missing, it either hallucinates an answer or gives up. The semantic layer fills that gap, supplying the business-domain context.

What the context layer is

A semantic layer is a set of declarative definitions of your metrics, how revenue is calculated, what an active customer is, what the numbers mean. Every consumer goes through the same definitions, so they all derive consistent, accurate results. But an agent that acts needs more than definitions of numbers. It needs to know what the things are, and what it may do to them. Those are three separate bodies of definition, and an agent needs all three.

The **domain model** says what exists. Entities, their relationships, and the meaning rules of the business: an order belongs to a customer, an active customer is one who purchased in the last ninety days. It gives the agent the vocabulary to interpret a request and plan against it. It is consulted, never executed; no query path to data runs through it.

The **semantic model** says how the numbers are computed. Metrics and dimensions, one versioned formula each, compiled to the same SQL every time and run against the analytical store. This is the semantic layer under a more exact name, and the job is to put correctness in the compiler rather than in the model's guess.

The **capability model** says what the agent may do. A curated set of operations against live systems, some that read (check payment status, retrieve a troubleshooting guide) and some that write (issue a refund). Each carries permissions and an owner, and the acting ones carry preconditions and a reversibility class as well.

Nouns, numbers, and verbs. Together they are the **context layer**, and what unites them is not that they are all about meaning, because the capability model plainly is not. It is that each one is a place where a guarantee is declared once, in version control, instead of being worked out afresh by the model on every request. The definitions are the layer; the interface, MCP today, is just the door.

A reader who works with dbt will object that its semantic models already declare entities, so why separate the domain model out. Because entities declared inside the metrics layer are scoped to metrics, and the capability model has to be written in the same vocabulary as the semantic one or the two drift apart. A refund acts on the same customer the revenue figure counts. One vocabulary underneath, or you get two.

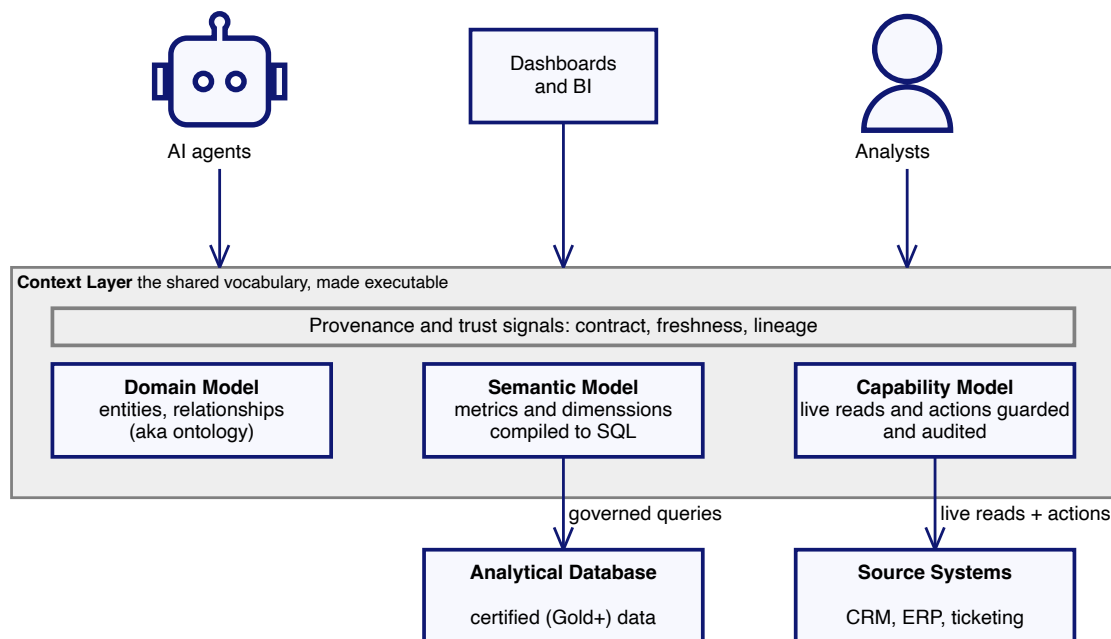


Figure 2: The context layer: a domain model of entities and relationships, a semantic model of metrics compiled to SQL against the analytical store, and a capability model of guarded reads and actions against live systems, with provenance signals across all three. The domain model has no arrow out because it is consulted rather than executed; the other two are written in its vocabulary. Dashboards and analysts reach the semantic model; agents are the first consumer to need all three, which is the shift this article is about.

All three models are code in source control. They go through code reviews, get tested in CI, and progress through environments before reaching production. When the definition of “revenue” or the rule on refunds changes, you change it in one place and it propagates everywhere. Agents never reach the underlying data directly; they go through the context layer, which constrains and governs both what they can ask for and what they can do.

Metrics as code

In practice, the business logic lives right in the definition, $\text{revenue} = \text{order_amount} - \text{discount_amount}$, not buried in a BI tool or an ad hoc SQL view. The agent receives a natural language question, and the semantic model resolves it to correct, constrained SQL. The agent doesn't guess table names or join paths; it uses the definition.

The examples here use dbt MetricFlow syntax (dbt is mid-migration from measures to a metrics-first spec; the widely-used form is shown here, and the concept holds either way). *Cube.js*, *Snowflake*, and *Databricks* all follow similar patterns. The tool matters less than the discipline of getting business logic into version controlled code.

```
semantic_models:
  - name: orders
    model: ref('orders')
    defaults:
      agg_time_dimension: order_date
    entities:
      - name: order_id
        type: primary
      - name: customer_id
        type: foreign
    dimensions:
      - name: order_date
        type: time
        type_params:
          time_granularity: day
    measures:
      - name: revenue
        agg: sum
        expr: order_amount - discount_amount
        create_metric: true
```

Same question, very different SQL

Let's consider an example. Ask “What was Q3 revenue for Product X?” of an agent *without* a semantic model, and it guesses at table names, uses the wrong column, has no fiscal-calendar mapping, and misses the join.

```
-- Before metric definition
SELECT SUM(amount)
  FROM sales_data
 WHERE product = 'Product X'
    AND quarter = 'Q3'
```

Ask the same question *with* a semantic model, and the agent is constrained to the correct table, the net-revenue formula from the YAML definition, the right fiscal-calendar dates, and the valid join path.

```
-- Constrained by metric definition
SELECT SUM(order_amount - discount_amount)
FROM orders o
JOIN products p
  ON o.product_id = p.id
WHERE p.name = 'Product X'
AND o.order_date
  BETWEEN '2025-07-01'
    AND '2025-09-30'
```

The semantic model doesn't make the agent smarter. It stops it from guessing. For an agent that acts on the answer unchecked, that's what matters.

How agents use it

Take the semantic model on its own, the path a quantitative question travels. End to end, the flow looks like this. The agent sends a natural-language question (step 1). The semantic model looks up metric definitions, valid dimensions, join paths, and access rules, via MCP (step 2), then generates constrained SQL (step 3), both inside the same component. The data warehouse executes the query (step 4). The result flows back to the agent with full lineage metadata (step 5).

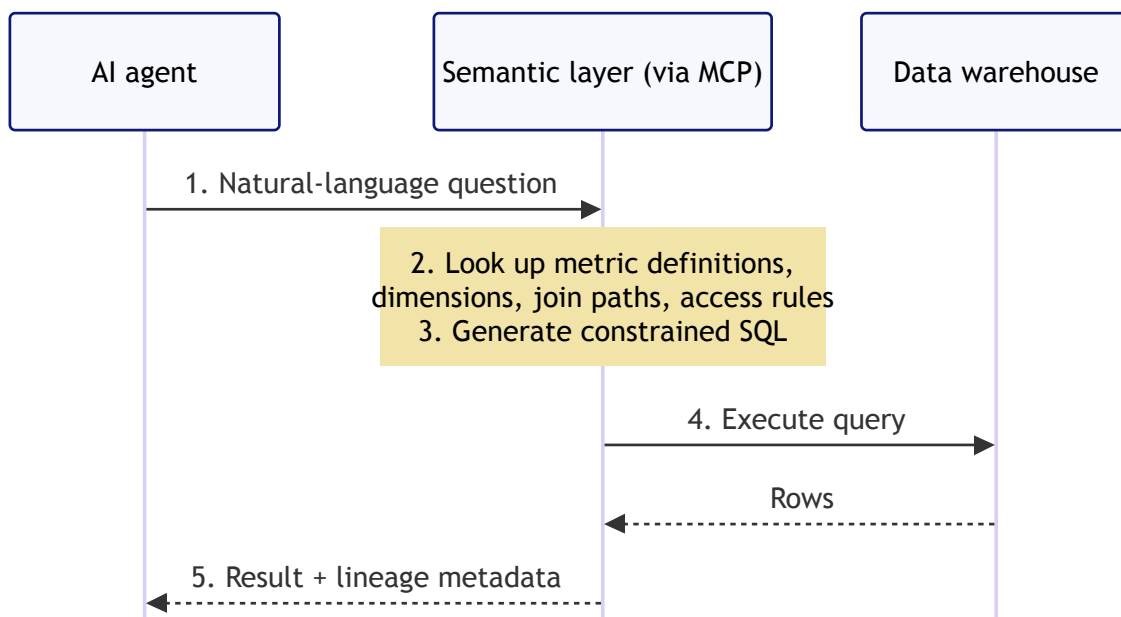


Figure 3: One of the three paths: a quantitative question answered through the semantic model. Questions about what things are go to the domain model, and reads or actions against live systems go through the capability model.

*Agents pick from governed metrics,
never raw tables they can misread*

The semantic model constrains what the agent can ask for. dbt's, for instance, dynamically surfaces only the dimensions applicable to the selected metrics, which prevents the agent from generating plausible sounding but incorrect queries. And that lineage metadata in step 5 is the foundation for the traceability we covered earlier. Context and traceability reinforce one another.

Where to start

The temptation with a context layer is to model the whole business before you ship anything. Resist it. Start with the semantic model, because the value is concentrated in a handful of metrics, the contested ones that mean different things to different teams. Let your first agent use case set the scope, and grow the domain model and the capabilities it actually needs rather than the ones you can imagine. A narrow, correct context layer beats a sprawling, half-agreed one.

- 1. Find your conflicting metric definitions. Most organizations have several definitions for their most important metrics, revenue being the classic, with its gross vs net, with or without returns variations. Those conflicts are your biggest agent risk and your quickest win.
- 2. Pick a tool, but focus on the discipline. Any mainstream semantic layer tool will do; what matters is the discipline behind it, metric definitions in version control, one agreed definition per metric, and agents querying through the layer, not the raw schema.
- 3. Route agents through the context layer, never the raw schema. The agent should see governed metrics and dimensions, not raw tables and joins. MCP is the common way to expose the layer today, and dbt, Cube, and AtScale all ship MCP servers, but the principle holds however you connect, the point is the abstraction, not the protocol.
- 4. Test adversarially. The best way to find gaps is adversarial testing, every hallucination points to a missing definition. Fix the definition, not the prompt. And don't boil the ocean, start with the metrics your first agent use case needs.

Traversing the domain model: knowledge graphs

The semantic model shines for structured metric queries such as “what was revenue by region.” But some agent tasks demand richer relationship reasoning across entities, events, and time. Consider a customer who bought Product X, then churned after a pricing change. A fixed number of hops like that is an ordinary join. What flat tables handle badly is traversal whose depth you don't know when you write the query, following a chain of relationships until you find what you are looking for. That is the domain model's territory, the entities and how they connect.

The common way to store and traverse that map is a knowledge graph, which is a storage choice for the domain model rather than a fourth thing to build. GraphRAG from Microsoft uses community detection to handle abstract queries that traditional RAG can't, and Graphiti builds temporally aware knowledge graphs for evolving facts. (Both sat at *Trial* on the Thoughtworks Radar as of 2026.) The semantic model still defines the metrics; the graph carries the connections between customers, products, events, and decisions over time. Together they give agents something close to institutional memory, the kind of knowledge that would take a new hire months to absorb.

Now agents have trusted data, governance, and context. But can they actually act?



From Searchable to Actionable: Agent-Ready Data Access

Once agents understand your data and governance is in place, the question shifts to access. How do agents reach the data and act on it? The answer is more than “RAG”. It's a full spectrum, from retrieval, to real-time queries, to controlled *write-back* actions. That whole spectrum is the capability model, the third of the three, and the write-back end is where its guardrails earn their keep.

Your agent can read, but it can't act

Let's take an example. An employee reports a PO (purchase order) issue. An ideal agent would do three things, retrieve the relevant troubleshooting guide, check whether the PO payment service is down right now, and create a help desk ticket if needed.

Traditional RAG, the pattern most organizations have deployed, only does step one. It searches documents and retrieves content. It can't query a live monitoring system to check service status, and it certainly can't create a ticket in ServiceNow or Jira. That gap between *searchable* and *actionable* is the subject of this final topic, and we will use the PO scenario to elaborate.

The data access spectrum

This framing comes from Microsoft's Cloud Adoption Framework for AI, which formalizes it as RAG + MCP-Read + MCP-Write.

1. Retrieval. RAG, vector search, document lookup. The agent finds relevant content. Most organizations live here today.
2. Real-Time Query. The agent queries live systems, checks service status, reads from databases in real time.
3. Write-Back. The most powerful and most dangerous tier. The agent creates tickets, updates records, triggers workflows.

Each step up the spectrum adds capability, and risk. The PO scenario maps cleanly across all three.

- Retrieve the guide (Retrieval)
- Check payment status (Real-Time Query)
- Create the ticket (Write-Back)

The shift to agentic AI requires all three, not just the retrieval most teams have built.

MCP has quickly become the default way to wire these tiers up, and its rise has been remarkably fast. But the mechanism matters less than the demarcation. What counts is keeping retrieval, real-time reads, and write-back as separate, deliberately governed levels of access, whether you expose them through MCP or your own native APIs.

Three primitives, one protocol

Agents reach all of this through MCP, the Model Context Protocol. Its primitives sit on a risk gradient, Resources (read-only) are safe, Prompts shape behavior, and Tools change state. That gradient maps straight onto the tiers, Resources to retrieval and Tools to write-back, which is why the safe path is to expose Resources first and graduate to Tools only under governance. In the PO scenario, Resources serve the troubleshooting docs, a Prompt guides triage, and Tools run `check_service_status()` and `create_support_ticket()`.

Antipattern: naive API-to-MCP conversion

How you design those Tools matters as much as when you reach for them. The common, costly mistake is to take existing REST APIs and wrap them one-to-one, so every endpoint becomes a tool. The result is *tool sprawl*, 50 tools with names like `get_po_payment_status`, `create_ticket_po_payment`, `create_ticket_po_payment_network`. The agent then has to choose among 50 barely-distinguished tools with little context, and LLMs are bad at that; accuracy drops sharply as the tool count climbs. The Thoughtworks Tech Radar put “naive API-to-MCP conversion” on **HOLD** for exactly this reason.

The better approach exposes the same functionality as a handful of well-designed capabilities with rich descriptions and parameterized inputs. `check_service_status` takes a service name and location, one tool for all services and all locations. `create_support_ticket` is parameterized with category, priority, and description. The descriptions are detailed enough for the LLM to know when to reach for each one.

Five to ten well described business capabilities will outperform 50 thin API wrappers almost every time

The principle is to *design capabilities, not endpoints*. Five to ten well described business capabilities will outperform 50 thin API wrappers almost every time. And this principle is protocol-agnostic, whether an agent reaches your data through MCP, through another agent, or through whatever standard comes next, the properties that make it agent-ready are the same, rich descriptions, parameterized access, clear schemas.

What a capability declares

A rich description tells the agent when to reach for a capability. It says nothing about whether the agent is allowed to, or what happens if it is wrong. That is the rest of the declaration. Every capability carries **permissions**, who may invoke it and acting as whom, and an **owner**, the person accountable when it misbehaves. The ones that act carry two more.

Preconditions are the conditions that must hold before the action may proceed, checked against live state at the moment of acting rather than against whatever the agent read earlier in its plan. A refund needs an original payment, not yet refunded, within the amount the invoking user may authorise.

Reversibility is the class of damage the action can do: cleanly reversible, reversible at a cost through some compensating transaction, or irreversible. This is the more useful predictor of safe autonomy than the money involved. A \$50,000 internal ledger correction you can back out is a safer thing to automate than a \$200 payment to an external account you cannot claw back. Where the staged autonomy ladder earlier keys its guardrails to transaction size, prefer keying them to reversibility, and let irreversible actions require human approval whatever stage the agent has reached.

*Reversibility predicts safe autonomy
better than the size of the transaction*

Which raises the question of where the rules in those preconditions come from, because most of them are written down in prose somewhere, in a refund policy, a contract, a compliance manual.

Retrieved text informs, it never gates

Business documents remain where the business writes its rules down. But a rule that gates an action must not be read and interpreted at the moment of acting. Rules are extracted from those documents ahead of time, curated by a human, and stored as declared preconditions in the capability model, each with a link back to the passage it came from.

At action time the agent may still read unstructured content, a complaint ticket, a contract clause, to work out what to propose. Only the declared rules decide what is permitted, and they are checked deterministically against live state. The boundary is between *informing* and *gating*. Retrieved text can shape what the agent suggests and

serve as evidence for a human approver, but it never carries the authority to authorise the action itself.

That boundary is also a security property. Removing retrieved text from the authorisation path means a poisoned document cannot grant an agent a permission it did not already have, which is a stronger claim than merely shrinking what a hijacked agent can reach. It is not a complete defence, because injected text can still influence what the agent proposes, and a human approver shown fabricated evidence may wave it through. What it removes is the path where the document authorises the action directly, with nobody in between.

The provenance link is what keeps the declarations honest as the documents move underneath them. Be careful what you promise here. Detecting that a document changed is easy; knowing that the change invalidated a precondition derived from it is a judgement, not a diff. What the link buys you is a review queue, the derived rules flagged for a human to re-check when their source moves, in the same spirit as keying a freshness SLA to when the index was last rebuilt rather than to when the content last appeared to change.

Where no declaration covers the situation, the agent does not improvise from its own reading of policy. It escalates. This is the hard gate from earlier in a different setting, the same instinct that says any contract or SLA breach forces a human rather than a lower score. An undeclared case degrades the agent to supervised, not to autonomous.

Extraction and curation is a pipeline like any other, and it needs an owner, a cadence, and somebody who clears the review queue. Which is the subject of a later section, because none of this maintains itself.

End to end: the PO payment scenario

With all three tiers in place, the PO issue we opened the section with runs end to end, the agent retrieves the troubleshooting guide (a read-only Resource), checks the live payment status (a Tool that reads), and files a ticket (a Tool that writes), all in a single workflow.

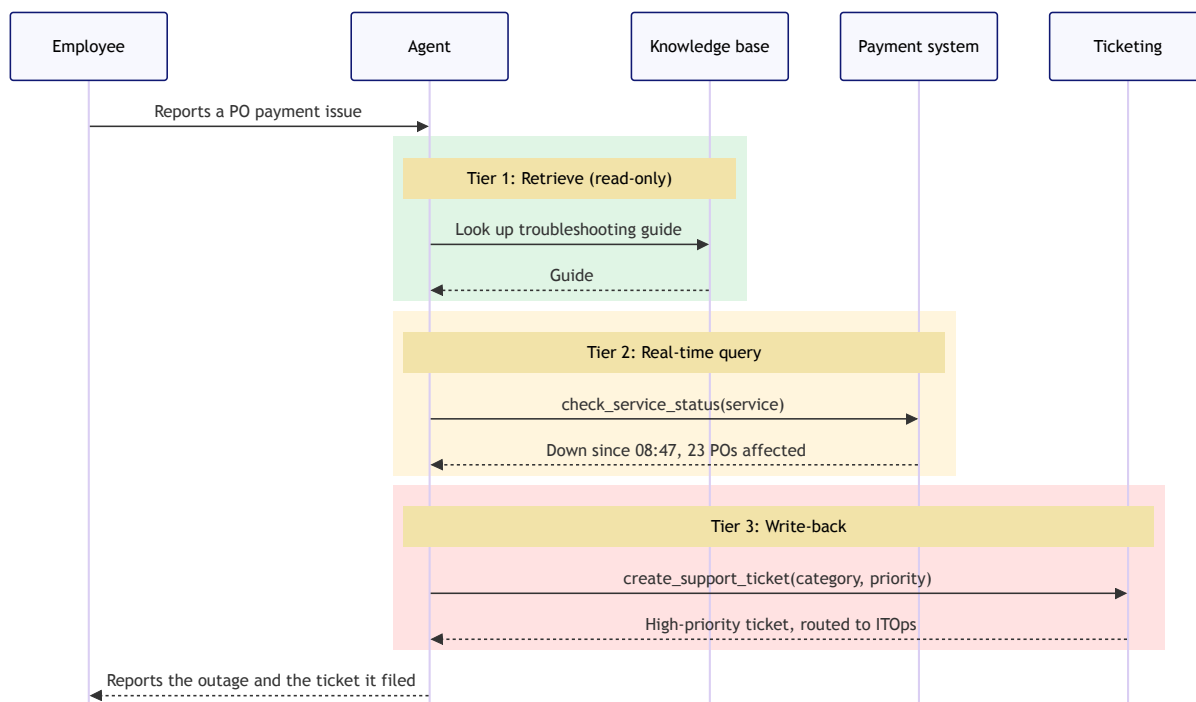


Figure 4: One agent, three tiers: retrieval, real-time query, then write-back, combined into a single response.

Done manually, the employee would wait in a queue, explain the issue, have a support agent check the monitoring dashboard, and get a ticket created. The agent is now able to do all this in one pass.

Where to start

The safe way in is to climb the tiers, not leap to write-back. Most teams already live in retrieval, the read-only tier where risk is lowest. Write-back is where the real danger sits. So earn your way up. Map what each use case needs, expose read-only access first, and add write-back last, only once you can log every action. Don't let the thrill of an agent that can act rush you past the steps that make acting safe.

- 1. Map your data access tiers. Take your top three agent use cases and classify what each needs, retrieval, real-time query, or write-back. Most gaps live in real-time query and write-back.
- 2. Design capabilities, not endpoints. Group existing APIs into 5–10 well-described business capabilities. Rich descriptions matter, they're what the LLM uses to decide which tool to call.
- 3. Start with MCP Resources. Read-only access is the lowest risk entry point. Expose knowledge bases, config data, and documentation as Resources. Graduate to Tools only once governance is in place.

- 4. Instrument from day one. Before deploying any agent with write access, log every tool invocation, who triggered it, what was called, when, and critically, on whose behalf. This feeds the audit trail from the Traceability and Governance section.

The AI-ready data stack

We've now walked through all four topics, contracts that make data trusted, a context layer that makes it meaningful and actionable, access patterns that let agents act on it, and observability that makes those actions auditable. Treated separately, they look like four work streams you could staff independently. But they aren't independent. They build on one another, and the order in which they're built matters.

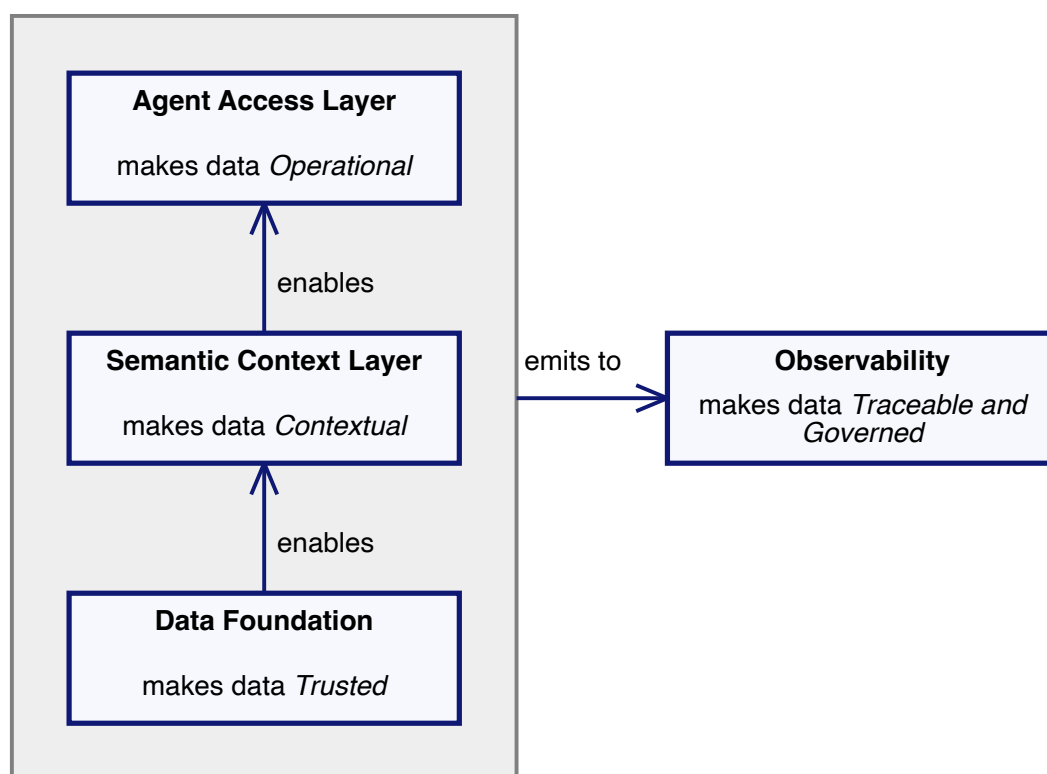


Figure 5: The AI-ready data stack: three dependent layers built bottom-up, with observability cutting across all of them from day one.

The dependencies run bottom-up. You can't attach meaning to data you can't trust, so context sits on the foundation. You can't safely let agents act without that meaning to constrain them, so access sits on context. Skip either of those and everything above it

collapses. That's exactly why so many agentic AI programs stall. They jump straight to agent access without building the foundation underneath. Observability is different. Rather than a fourth tier stacked on top, it runs alongside all three. Every layer has to be traceable and auditable from the moment it handles real work. The trust checks, the semantic queries, the agent's actions, all of it has to be explainable in production, not whenever you get around to instrumenting it. It is also much harder to retrofit onto a running system than to build in from the start. Either way, you wire it in from day one.

Who owns all this?

The stack has one more dependency the diagram can't draw. Every layer in it produces an artifact that has to be kept true, a data contract, a metric definition, an access scope, an observability trace. Artifacts don't maintain themselves. A contract with no owner drifts out of sync with the source it describes. A definition of “revenue” with no owner forks back into the three conflicting versions you just consolidated. An access scope with no owner quietly widens until it's a standing service account again. The technology is necessary, but it's the operating model that keeps it honest.

The discipline that makes this work is treating data as a product. Each dataset, contract, and metric has a named owner, a published contract and SLA, and a versioned lifecycle, the same way an API does. You won't always know every consumer, and for public or broadly shared data you can't, which is precisely why the contract matters, it's the stable promise unknown consumers build on, and a deprecation policy is how you change it without breaking them. When the `product_pricing` contract blocks a deployment at 2 a.m., someone is accountable for it. When finance and sales disagree on “revenue,” someone owns the decision. When a new agent asks for access, someone owns the scope and reviews it. These aren't infrastructure questions; they're ownership questions, and no tool answers them for you.

A human consumer of an unowned, drifting dataset notices and works around it. An agent consumes it at machine speed and scale, and propagates the error just as fast. The faster and more autonomous your consumers, the less you can afford data without an owner.

Where do you stand?

Before deciding what to build, it helps to locate yourself. Score each attribute against the signals below, all drawn from the topics above.

Attribute	Human-era	In Transition	Agent-ready
Trusted	Loose schemas, no freshness SLAs; quality rests on an analyst noticing when a number looks off	Contracts on a few critical datasets; quality checked but not enforced in CI/CD.	Contracts enforced as code, freshness SLAs per consumer, quarantine before agent storage, agents read Gold only (tables and embeddings)
Contextual	Metric definitions live in BI tools, SQL, and people's heads; humans supply the context	Some metrics defined as code, but definitions still conflict and agents may still hit the raw schema	A context layer in Git: entities and relationships in a domain model, one semantic definition per metric, and a curated set of capabilities; agents route through it, never the raw schema
Traceable	Logs show what a person queried and when; the why lives in the analyst's head	Traces on some agent workflows; reasoning captured inconsistently	Every agent workflow emits traces with spans, reasoning, and sources; any decision's "why" is reconstructable
Governed	People access data through their own roles; systems share broad service accounts	Agents run on scoped but long-lived, coarse credentials	Delegated per-user access, just-in-time credentials, least privilege; lethal-trifecta paths closed
Operational	No agent acts on the data; people read dashboards and take actions by hand	Agents retrieve via RAG; real-time reads emerging; write-back experimental or ungoverned	All three tiers via well-designed capabilities; write-back gated by staged autonomy and instrumentation

Don't average the rows, because the stack is dependency ordered, your readiness is capped by your weakest foundational layer, a flawless context layer sitting on untrusted data is still not agent ready. Find your weakest row, and that's where the next investment goes.

Four things to start on

Each topic came with its own starting points. Treat those as tactical checklists for the work itself. The four below are where to start. The first, instrumenting from day one, isn't a build-order step. It runs alongside everything else, which is why it comes first and never stops. The other three build from the bottom of the stack up, because you're only as ready as your weakest foundational layer. The highest-leverage single move among them is the context layer, since context moves accuracy further than a bigger model does, but it only pays off once the data beneath it can be trusted. Build up to it.

1. Instrument from day one. This isn't a step in the sequence so much as a constant that runs under all of them. Put traces and spans in every workflow from the start, because observability is far harder to retrofit than to build in, and you'll want audit trails that answer “why” for debugging today and regulators tomorrow.
2. Contract everything. Freshness SLAs, strict schema enforcement, quarantine for bad data. This is the floor the rest stands on, agents can't smell bad data, so the data architecture has to smell it for them.
3. Context over models. Once the data can be trusted, a context layer is the highest-return thing you can build on top of it. Its semantic model alone carries the point: in AtScale's text-to-SQL benchmark, accuracy jumped from under 20% on the raw schema to over 92.5% with a semantic layer, on the *same model*.
4. Read before write. Start with MCP Resources (read-only) and graduate to Tools (write) only with governance in place. Earn autonomy in stages, shadow mode, then supervised, then autonomous with guardrails.

When agents become the primary consumers of your data, your data architecture becomes your AI architecture.

We go much deeper on all of this, and on the broader operational and analytical data architecture decisions around it, in our forthcoming O'Reilly book, [Data Architecture for Software Architects](#).

Acknowledgments

Thanks to Martin Fowler for encouraging to write about this topic based on a series of talks we gave at multiple conferences, we would also like to thank Rebecca Parsons, Kevin Hartman, Paul Hammant, Arun Srinivasan, Swapnil Phulse, Brian Smith, Ramanathan Santhanam, Cameron Casher and Mark Taylor for giving us a thorough review and actionable comments to improve the article

Like most things in the industry, we used AI assistance to help research, organize, and format some parts of our writing.

► Significant Revisions

/thoughtworks



© Martin Fowler | Disclosures